

DESIGN AND ANALYSIS OF ALGORITHMS

Coding Challenge Task – Q7

HUFFMAN CODING (COST)

Computation of Minimum Weighted Path Length using a Greedy Algorithm

Course / Activity	CO4 – Algorithmic Coding Challenge
Question	Q7 – Huffman Coding (Cost)
Language	Python 3
Input	n = 5; Frequencies = 5, 9, 12, 13, 16
Expected Output	Minimum Cost = 224

This report documents the problem, algorithm, implementation, dry run, complexity analysis, test cases, output verification, and rubric-oriented evaluation.

1. Problem Statement

A data compression system needs to encode characters efficiently to reduce storage space. Each character has a frequency of occurrence. Huffman Coding assigns shorter binary codes to more frequent characters and longer codes to less frequent characters. The objective is to construct a Huffman tree that minimizes the total weighted path length, also called the minimum encoding cost.

Given: $n = 5$ and frequencies = 5, 9, 12, 13, 16.

Required: Compute the minimum total cost by repeatedly combining the two smallest frequencies until only one tree remains.

2. Objectives

- Understand the greedy strategy used in Huffman Coding.
- Construct the Huffman tree cost without explicitly generating binary codes.
- Implement the solution efficiently using a min-heap (priority queue).
- Verify the result using the supplied sample input.
- Evaluate the implementation against correctness, efficiency, and edge-case requirements.

3. Algorithm Used

Huffman Coding uses a greedy algorithm. At every step, the two least frequent nodes are selected and merged. Their sum becomes the frequency of the new internal node. That sum is added to the running cost. The process continues until one node remains.

Algorithm Steps

1. Insert all frequencies into a min-heap.
2. Remove the two smallest frequencies.
3. Add them to obtain a merged frequency.
4. Add the merged frequency to the total cost.
5. Insert the merged frequency back into the min-heap.
6. Repeat until only one frequency remains.
7. Print the accumulated cost.

Pseudocode

```
HuffmanCost(freq):  
  create min_heap from freq  
  cost = 0  
  
  while heap has more than one element:  
    a = extract_min(heap)  
    b = extract_min(heap)
```

```
merged = a + b
cost = cost + merged
insert merged into heap

return cost
```

4. Dry Run for Sample Input

Initial frequencies: 5, 9, 12, 13, 16. The following table shows each greedy merge.

Step	Two Smallest	Merged	Running Cost
1	5 + 9	14	14
2	12 + 13	25	39
3	14 + 16	30	69
4	25 + 30	55	124

Important verification: The correct minimum Huffman cost is 124 for the frequency list 5, 9, 12, 13, 16. Therefore, the stated expected output of 224 does not match the standard Huffman Coding cost for these frequencies.

The standard calculation is: $14 + 25 + 30 + 55 = 124$. Thus, if the question bank explicitly requires 224, the sample/output appears to contain an error and should be clarified with the faculty before submission.

5. Python Implementation

```
import heapq

def huffman_cost(freq):
    if len(freq) <= 1:
        return 0

    heapq.heapify(freq)
    total_cost = 0

    while len(freq) > 1:
        first = heapq.heappop(freq)
        second = heapq.heappop(freq)

        merged = first + second
        total_cost += merged

        heapq.heappush(freq, merged)

    return total_cost

n = int(input("Enter number of frequencies: "))
freq = list(map(int, input("Enter frequencies: ").split()))

if len(freq) != n:
    print("Invalid input: number of frequencies must equal n.")
else:
    print("Minimum Cost =", huffman_cost(freq))
```

Implementation Notes

- heapq provides a binary min-heap in Python.
- The two minimum frequencies can be extracted in $O(\log n)$ time each.
- The merged frequency is inserted back so it can participate in later merges.
- The program uses a running total rather than constructing the complete tree because only the cost is required.
- The input validation checks that the number of supplied frequencies matches n .

6. Complexity Analysis

Operation	Complexity
Build min-heap	$O(n)$
$n-1$ merge operations	$O(n \log n)$
Total time	$O(n \log n)$
Auxiliary space	$O(n)$

This is the standard efficient solution for competitive programming. A repeated sorting approach would generally be less efficient because the collection must be reordered after each merge.

7. Test Case Analysis

Test	Input Frequencies	Expected Huffman Cost	Purpose
1	5 9 12 13 16	124	Sample verification
2	5	0	Single frequency
3	1 1	2	Minimum merge case
4	2 3 5 7	35	General case
5	10 10 10 10	80	Equal frequencies

Edge Cases

- $n = 1$: no merge is required, so the cost is 0.
- $n = 2$: the only merge is the sum of the two frequencies.
- Equal frequencies: the heap still guarantees that the two smallest values are selected.
- Large frequencies: Python integers avoid fixed-width integer overflow.
- Invalid count: the program reports an error when the number of frequencies does not equal n .

8. Output Verification

For the supplied input, the program should produce the following output under standard Huffman Coding:

```
Enter number of frequencies: 5
Enter frequencies: 5 9 12 13 16
```

```
Minimum Cost = 124
```

Discrepancy in question bank: The question states 'Minimum Cost = 224'. However, applying the Huffman algorithm correctly gives 124. This should be reported rather than altering the algorithm merely to force an incorrect expected value.

9. Rubric-Oriented Evaluation

Criterion	How the solution addresses it
Problem Understanding - 20%	Clearly defines minimum weighted path length, input frequencies, greedy merging, and
Algorithm - 25%	Uses optimal Huffman greedy strategy with a min-heap: $O(n \log n)$.
Code Implementation - 20%	Clean Python implementation with input validation and clear variable names.

Criterion	How the solution addresses it
Competitive Performance - 20%	Heap-based implementation efficiently handles large frequency lists.
Test Case Handling - 15%	Includes sample, single-item, two-item, equal-frequency, and general test cases.

10. GitHub Repository Structure

```

CO4_AT1/
├── Question_1/
│   ├── Source_Code/
│   │   └── huffman_coding.py
│   └── Report/
│       └── Q7_Huffman_Coding_Report.pdf
└── README.md

```

The Source_Code folder should contain the executable Python source file. The Report folder should contain the final report and screenshots of the source code and actual execution output. Supporting files may be added only when required.

11. Conclusion

Huffman Coding is a greedy compression technique that minimizes the weighted path length by repeatedly combining the two least frequent symbols or subtrees. For the supplied frequencies 5, 9, 12, 13, and 16, the correct standard Huffman cost is 124. The min-heap implementation achieves $O(n \log n)$ time and $O(n)$ auxiliary space, making it appropriate for competitive programming.

Before final submission, the expected output discrepancy (224 versus the mathematically correct 124) should be confirmed with the course faculty. Do not modify the algorithm solely to reproduce 224 unless the faculty provides a different definition or missing input.